

Algorithme Des Lucioles

Une méthode d'optimisation inspirée de la nature



المدرسة الوطنية للعلوم التطبيقية - القنيطرة
National School of Applied Sciences - Kenitra



Algorithme Des Lucioles

Une méthode d'optimisation inspirée de la nature

Année universitaire 2024 - 2025



المدرسة الوطنية للعلوم التطبيقية - القنيطرة

National School of Applied Sciences - Kenitra

Notre Encadrante

- Prof. Dr. Hanaa HACHIMI

Notre Equipe

- Zineb EL-GAOUT
- Fatima BOUHO
- Zainab KINALI
- Douaa QAFZ
- Soukaina MOUNA

Introduction

Les lucioles utilisent la lumière pour communiquer, se synchroniser et se protéger des prédateurs. Ce phénomène a inspiré des modèles informatiques, notamment l'algorithme des lucioles.



Aperçu

Que trouve t'on dans cette présentation ?

- 1.** Pourquoi cet algorithme ?
- 2.** Principe de fonctionnement des lucioles dans la nature
- 3.** Applications dans divers domaines
- 4.** Avantages et limites
- 5.** Algorithme des lucioles
- 6.** Trois applications Algorithme des lucioles
- 7.** Comparaison de l'Algorithme des lucioles avec les autres algos bio-inspirés

**Pourquoi Cet
Algorithme ?**





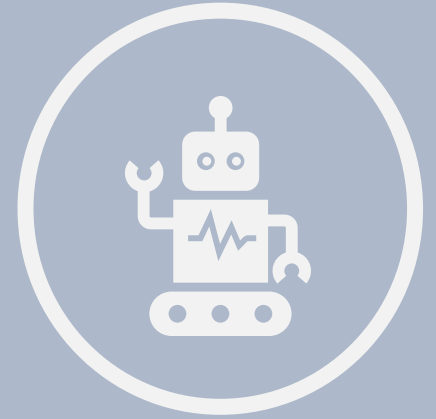
**Améliorer la
logistique
et la
production**



**Optimiser les
réseaux**



**Optimiser les
transports**



**Piloter
l'intelligence
artificielle et la
robotique**

Principe de fonctionnement

1

Attraction

Les lucioles sont attirées par les plus lumineuses.

2

Déplacement

Une luciole se déplace vers une plus brillante en ajustant sa position.

3

Exploration

Si aucune luciole plus lumineuse n'est détectée, elle explore aléatoirement.

Applications Dans Divers Domaine





Planification et ordonnancement

maintenance prédictive,
gestion des ressources



Logistique et transport

optimisation des itinéraires ,
réduction des coûts



Réseaux intelligents

gestion des capteurs,
réduction des interférences



Avantages Vs Limites

Optimisation efficace : Il explore plusieurs zones en même temps pour trouver la meilleure solution.

Attraction naturelle : Les lucioles vont automatiquement vers les meilleures solutions.

Exécution en parallèle : Chaque luciole peut être calculée à part, ce qui gagne du temps.

Grande adaptabilité : Il peut résoudre plusieurs types de problèmes : continus, discrets ou mixtes.

Résistance aux erreurs : Il continue de fonctionner même si les données sont incomplètes ou incorrectes.



Avantages



Limites

Dépendance aux paramètres : Il faut bien choisir les paramètres, sinon les résultats seront mauvais.

Convergence trop rapide : Il peut s'arrêter trop tôt sur une solution moyenne.

Moins efficace pour les gros problèmes : Quand il y a trop de variables, il devient lent et confus.

Trop de lucioles = lenteur : Le temps de calcul devient long quand il y a beaucoup de lucioles.

Ne gère pas bien les règles : Il faut ajouter des méthodes pour respecter les contraintes du problème.

Algorithme des lucioles



Les règles de base de l'algorithme

1

Toutes les lucioles s'attirent les unes aux autres, indépendamment de leur sexe ;

2

L'attractivité est proportionnelle à la luminosité

3

S'il n'existe pas de luciole plus brillante qu'une luciole donnée, celle-ci se déplacera aléatoirement dans l'espace.

1. La fonction objectif et la luminosité

Type d'optimisation	Fonction objectif $f(x)$	Luminosité $I(x)$
Minimisation	À rendre la plus petite	$I(x) = 1 / (f(x) + \epsilon)$
Maximisation	À rendre la plus grande	$I(x) = f(x)$

petit nombre pour éviter la
division par zéro.

Fonction de fitness

Meilleure fitness locale

la meilleure solution qu'une
luciole a trouvée jusqu'à un
point.

Meilleure fitness globale

la meilleure solution trouvée par
toutes les lucioles à travers
toutes les itérations.

2. Paramètres principaux de l’algorithme des lucioles

Paramètre	Symbole	Rôle
Nombre de lucioles	n	Nombre total de solutions (ou individus) dans la population.
Attractivité initiale	β_0	Détermine la capacité d'une luciole à attirer une autre au départ.
Facteur d’absorption de lumière	γ	Contrôle la diminution de l’attractivité avec la distance. Plus γ est grand, plus la lumière s’atténue vite.
Coefficient aléatoire	α	Intensité du mouvement aléatoire (exploration de nouvelles zones).
Distance entre lucioles	r_{ij}	Mesure la distance euclidienne entre deux lucioles x_i et x_j .

2. Paramètres principaux de l'algorithme des lucioles

2.1. Intensité lumineuse

$$I(r) = I_0 \cdot e^{-\gamma r^2}$$

$I(r)$ = intensité perçue à une distance r

I_0 = intensité à la source (souvent égale à la valeur de la fonction objectif)

2.2. Distance entre deux lucioles r

Pour chaque paire de lucioles (i,j)
la **distance euclidienne** entre x_i et x_j

$$r_{ij} = \|x_i - x_j\| = \sqrt{\sum_{k=1}^d (x_{i,k} - x_{j,k})^2}$$

la dimension du problème

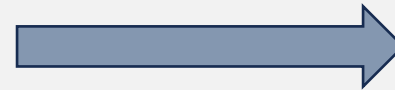
2. Paramètres principaux de l'algorithme des lucioles

2.3. Attractivité de la luciole

$$\beta(r) = \beta_0 \cdot e^{-\gamma r^2}$$

Attractivité à une
distance r

Attractivité Maximale
À $r=0$



détermine à quel point une
luciole est attirée par une
autre plus brillante.

3. Equation de mouvement des lucioles entre t et $t+1$

À chaque itération t , chaque **luciole** i se déplace vers les **lucioles plus brillantes** j suivant l'équation suivante :

$$x_i(t+1) = x_i(t) + \underbrace{\beta_0 e^{-\gamma r_{ij}^2} (x_j(t) - x_i(t))}_{\text{Exploitation}} + \underbrace{\alpha \cdot (\text{rand} - 0.5)}_{\text{Exploration}}$$

Facteur d'aléa
(pour l'exploration)

Valeur aléatoire dans
l'intervalle $[0,1]$

Exploitation

Se déplacer vers la luciole
brillante avec $\beta(r)$

Exploration

Introduire une variation
aléatoire avec $\alpha \cdot (\text{rand} - 0.5)$

Les étapes de l'algorithme

1

Initialisation

Générer une population de lucioles avec des positions aléatoires.

2

Évaluation

Calculer la luminosité de chaque luciole via la fonction objectif.

3

Mouvement

Chaque luciole se déplace vers les lucioles plus brillantes

4

Mise à jour

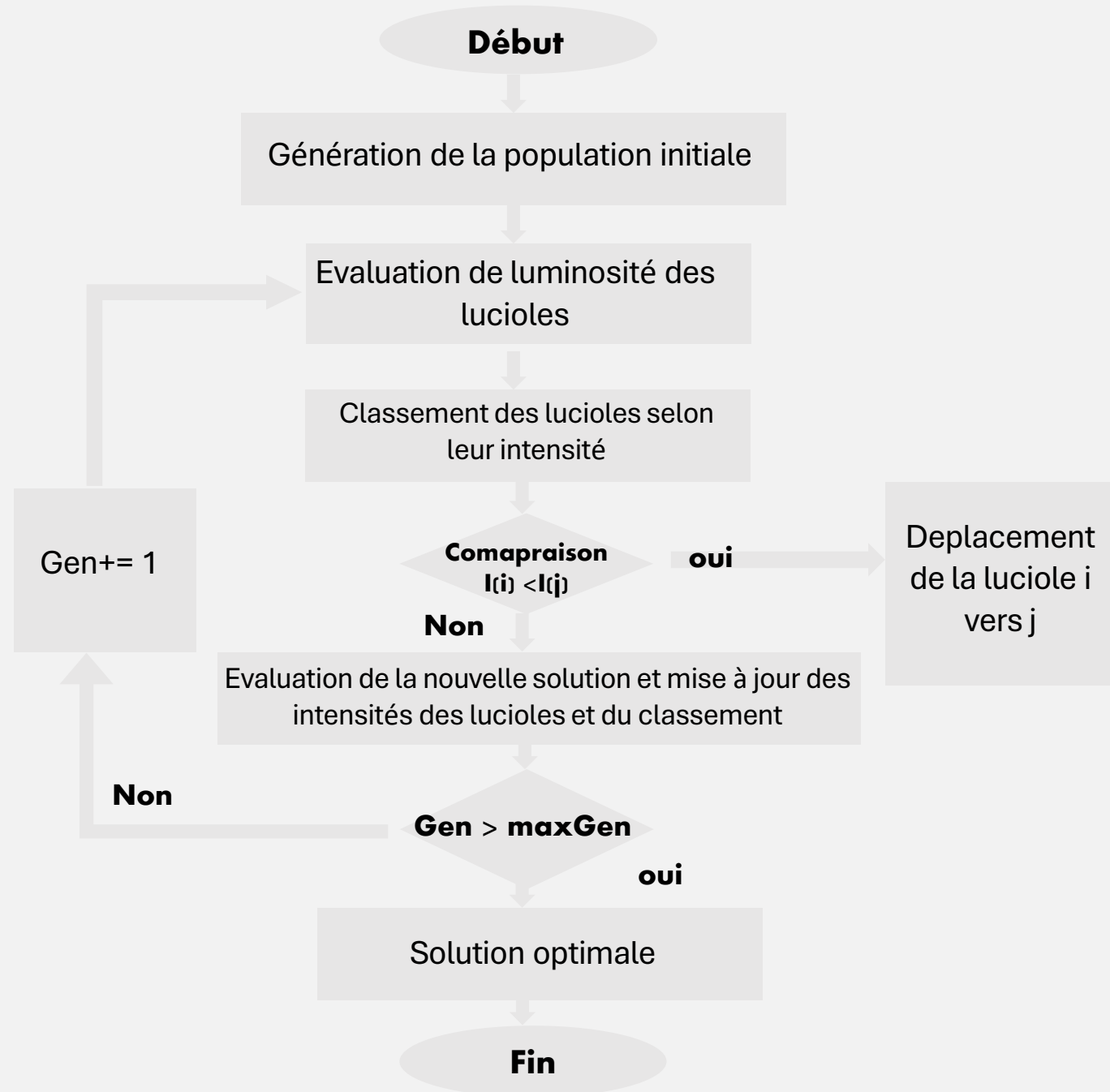
Recalculer les nouvelles positions et la fonction objectif.

5

Boucle

Répéter jusqu'à un critère d'arrêt

L'organigramme de l'algorithme des lucioles



Pseudo code

Début

1. Fonction objectif : $f(x)$, où $x = (x_1, x_2, \dots, x_d)$;
2. Générer une population initiale de lucioles x_i ($i = 1, 2, \dots, n$);
3. Formuler l'intensité lumineuse I de manière à ce qu'elle soit associée à $f(x)$;
4. Définir le coefficient d'absorption γ ;

Tant que ($t < \text{MaxGeneration}$)

 Pour $i = 1 : n$ (pour toutes les n lucioles)

 Pour $j = 1 : n$ (pour toutes les n lucioles)

 Si ($I > J$),

 Modifier l'attractivité en fonction de la distance r via $e^{(-\gamma r)}$;

 Évaluer les nouvelles solutions et mettre à jour l'intensité lumineuse;

 Fin si

 Fin Pour j

 Fin Pour i

 Classer les lucioles et trouver la meilleure solution actuelle;

Fin Tant que

résultats et visualisation;

Fin

Code en langage python

```
import numpy as np
import matplotlib.pyplot as plt

# Fonction objectif à maximiser
def fonction_objectif(x):
    |   return -np.sum(x**2) + 10

# Paramètres
n = 40 # Nombre de lucioles
d = 2 # Dimension de l'espace de recherche (2D)
generations = 100 # Nombre de générations (itérations)
alpha = 0.2 # Facteur d'exploration (bruit aléatoire)
beta0 = 1 # Attractivité maximale
gamma = 1 # Coefficient d'absorption (contrôle la rapidité de l'atténuation de l'attractivité avec la distance)
lower_bound = -5 # Limite inférieure des positions des lucioles
upper_bound = 5 # Limite supérieure des positions des lucioles
|

# Initialisation
population = np.random.uniform(lower_bound, upper_bound, (n, d))
intensities = np.apply_along_axis(fonction_objectif, 1, population)

best_scores = [] # Pour enregistrer le meilleur score à chaque génération
```

$$f(x) = -\sum_{i=1}^d x_i^2 + 10$$

Boucle principale

```
# Boucle principale
for t in range(generations):
    for i in range(n):
        for j in range(n):
            if intensities[j] > intensities[i]: # Maximisation
                r = np.linalg.norm(population[i] - population[j]) # Distance entre les lucioles
                beta = beta0 * np.exp(-gamma * r**2) # Attractivité selon la distance
                rand = alpha * (np.random.rand(d) - 0.5) # Composante d'exploration aléatoire
                population[i] = population[i] + beta * (population[j] - population[i]) + rand # Mise à jour de la position
                population[i] = np.clip(population[i], lower_bound, upper_bound) # Clipping pour ne pas sortir des limites
                intensities[i] = fonction_objectif(population[i]) # Mise à jour de l'intensité

        best_scores.append(np.max(intensities)) # On enregistre la meilleure intensité de chaque génération
```

Résultat

```
# Résultats
best_index = np.argmax(intensities) #L'indice de la luciole ayant la meilleure intensité (fonction objectif la plus élevée)
best_solution = population[best_index] #La position (solution) de la meilleure luciole.
best_value = intensities[best_index] #La valeur de la fonction objectif pour la meilleure solution.

print("✅ Meilleure solution trouvée :", best_solution)
print("❌ Valeur maximale de la fonction objectif :", best_value)
```

Sortie

```
✅ Meilleure solution trouvée : [-5.39836118e-05 -2.80974560e-04]
❌ Valeur maximale de la fonction objectif : 9.999999918139066
```

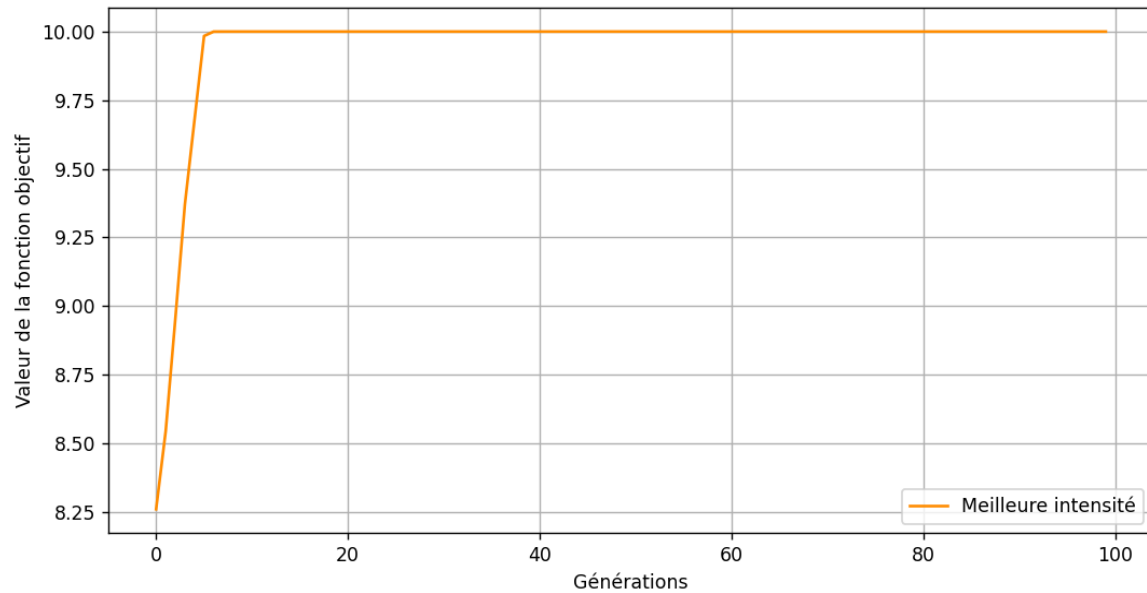
Visualisation des résultats

```
# 📊 Visualisation : évolution de la meilleure intensité
plt.figure(figsize=(10, 5))
plt.plot(best_scores, label="Meilleure intensité", color='darkorange')
plt.xlabel("Génération")
plt.ylabel("Valeur de la fonction objectif")
plt.title("Évolution de la meilleure solution")
plt.grid(True)
plt.legend()
plt.show()

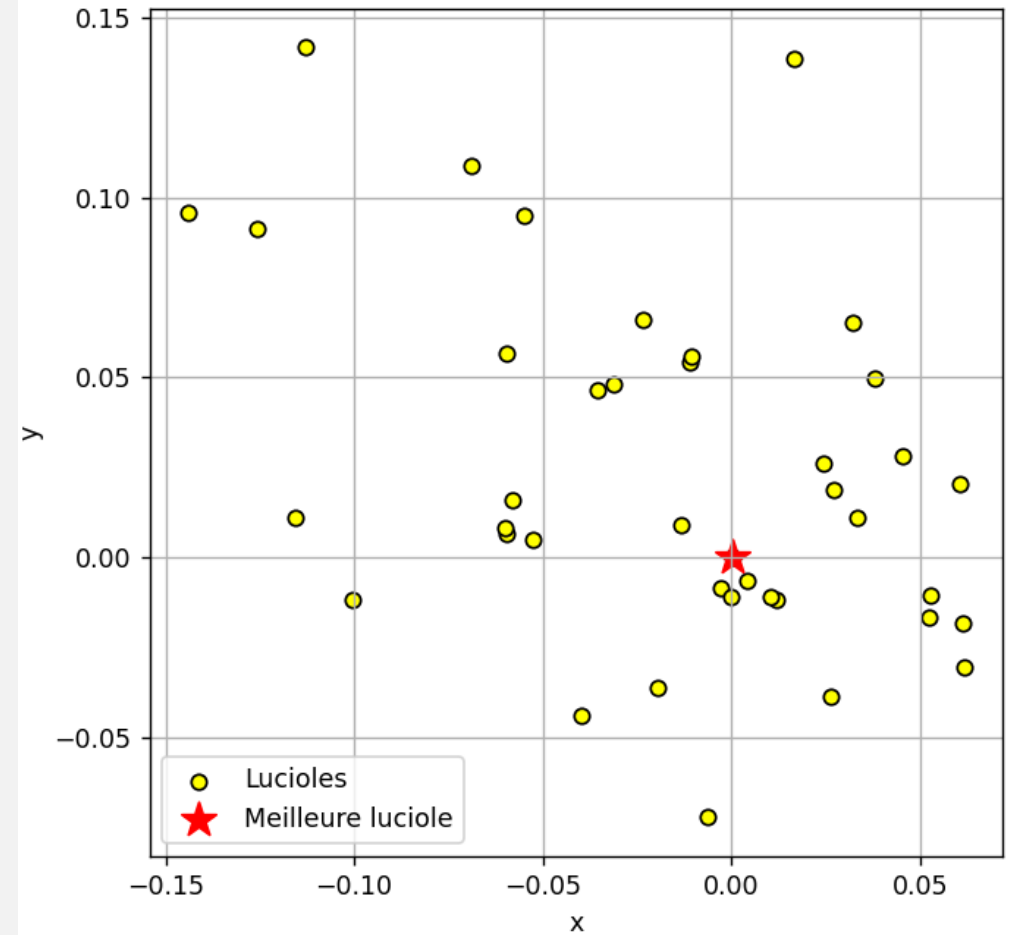
# 📍 Visualisation finale des lucioles si d = 2
if d == 2:
    plt.figure(figsize=(6, 6))
    plt.scatter(population[:, 0], population[:, 1], c='yellow', edgecolors='black', label='Lucioles')
    plt.scatter(best_solution[0], best_solution[1], c='red', marker='*', s=200, label='Meilleure luciole')
    plt.xlabel("x")
    plt.ylabel("y")
    plt.title("Position finale des lucioles")
    plt.legend()
    plt.grid(True)
    plt.show()
```

Visualisation des résultats

Évolution de la meilleure solution



Position finale des lucioles



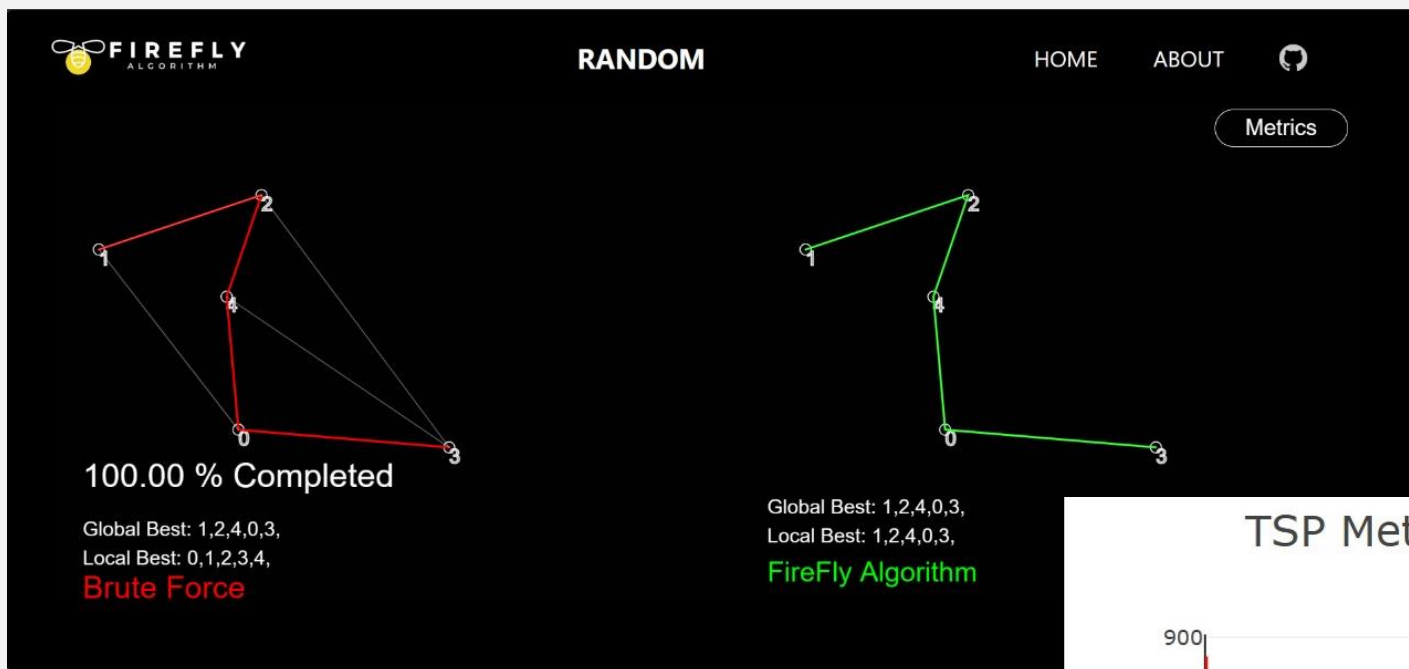
Applications de l'algorithme des lucioles



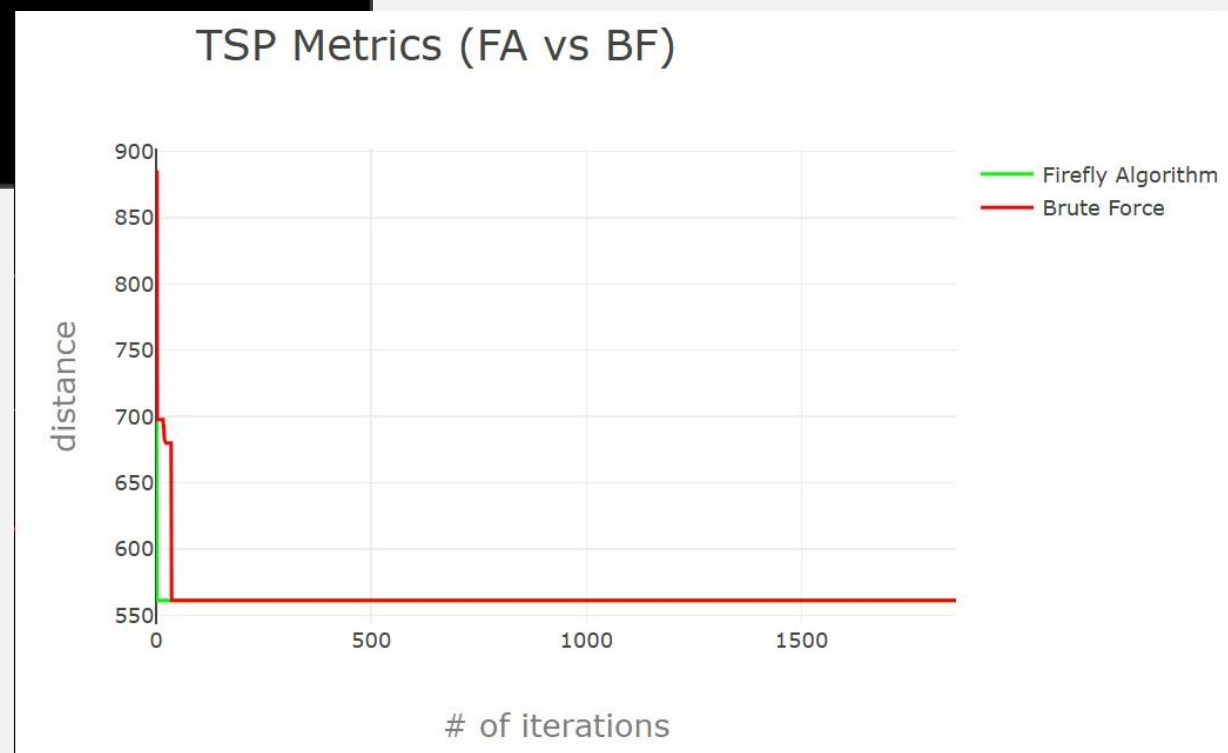
1ère Application

Voyageur de commerce

A screenshot of the 'RANDOM CITIES' configuration page on the Firefly Algorithm website. The page has a dark theme with white text and input fields. The 'FIREFLY ALGORITHM' logo is in the top left, and navigation links 'HOME', 'ABOUT', and a GitHub icon are in the top right. The main content area contains five input fields with labels and hints: 'Total Cities' (hint: 'keep the cities less than 30 (Performance Issues)', value: '10'), 'Population' (hint: 'keep the population less than 500 (Performance Issues)', value: '100'), 'Gamma' (value: '0,03'), 'Beta' (value: '1'), and 'Total Moves' (hint: 'keep the moves less than 100 (Performance Issues)', value: '20'). At the bottom, there is a 'Genetic Crossover' toggle switch, which is currently turned off.



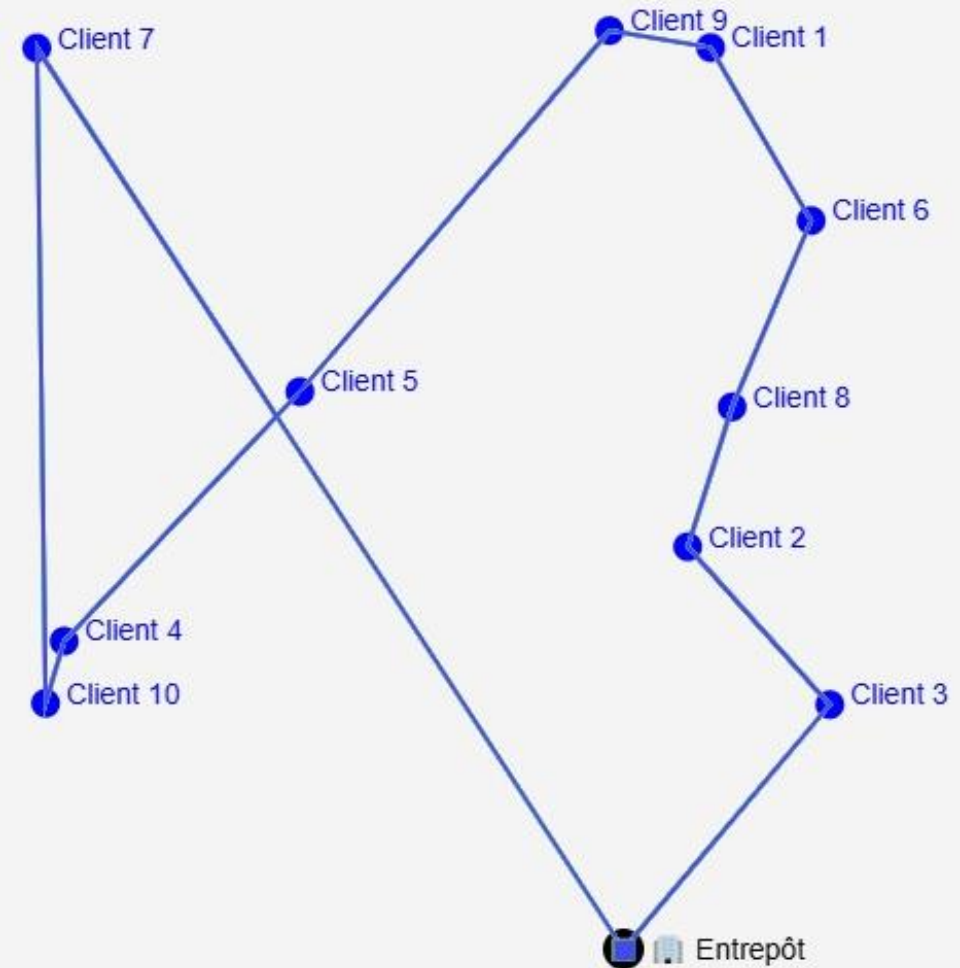
Resultat



2^{ème} Application

Tournée De Livraison

Cette application simule l'utilisation de l'algorithme de luciole pour optimiser une tournée de livraison. Des véhicules (représentés par des lucioles) partent d'un entrepôt pour livrer des colis à différents clients. L'algorithme trouve l'itinéraire optimal en utilisant la méthode du plus proche voisin. Les véhicules suivent cet itinéraire tout en explorant aléatoirement la zone, puis retournent à l'entrepôt après avoir livré les colis. L'application visualise en temps réel ce processus sur une carte.



3^{ème} Application

Emploi Du Temps

L'application optimise un emploi du temps en utilisant l'algorithme de luciole. L'utilisateur entre ses cours et ses créneaux horaires disponibles. L'algorithme génère des emplois du temps aléatoires, puis améliore ces solutions par itérations. Les lucioles (solutions) se déplacent vers la meilleure option en fonction des disponibilités. Le résultat final est un emploi du temps optimisé, affiché sous forme de tableau. Cela permet une meilleure gestion du temps et des ressources disponibles.

Entrez vos Cours

Cours 1

Titre du Cours:

chimie

Durée (en heures):

2

Cours 2

Titre du Cours:

statistique

Durée (en heures):

2

Ajouter un Cours

Ajouter un Cours

Votre Disponibilité

Disponibilité 1

Jour:

Mardi

Heure de Début:

10h

Heure de Fin:

15h

Disponibilité 2

Jour:

Lundi

Heure de Début:

8h

Heure de Fin:

9h

Emploi du Temps Optimisé

Emploi du Temps Optimisé

Heure	Lundi	Mardi	Mercredi	Jeudi	Vendredi
8h - 9h					
9h - 10h				chimie	
10h - 11h					
11h - 12h		statistique			
12h - 13h					
13h - 14h					
14h - 15h			algebre		
15h - 16h					
16h - 17h					
17h - 18h					

Comparaison de l'algorithme des lucioles avec d'autres algorithmes



FA vs PSO

FA vs GA

Les trois algorithmes sont testés sur des fonctions standards avec une même population ($n = 40$). Chaque test est répété **100 fois** pour garantir une analyse statistique fiable.

Les simulations sont faites sous Matlab avec un critère d'arrêt basé sur une variation très faible (10^{-5}).

Les résultats incluent : nombre moyen d'évaluations, écart type et taux de réussite.

Function	GA	FA
Michalewicz	89325 ± 7914 (95%)	2889 ± 719 (100%)
Rosenbrock	55723 ± 8901 (90%)	6040 ± 535 (100%)
De Jong	25412 ± 1237 (100%)	5657 ± 730 (100%)
Schwefel	227329 ± 7572 (95%)	7923 ± 524 (100%)
Ackley	32720 ± 3327 (90%)	4392 ± 2710 (100%)
Rastrigin	110523 ± 5199 (77%)	12075 ± 3750 (100%)
Easom	19239 ± 3307 (92%)	6082 ± 1690 (100%)
Griewank	70925 ± 7652 (90%)	10790 ± 2977 (100%)
Yang	37079 ± 8920 (88%)	5152 ± 2493 (100%)
Shubert	54077 ± 4997 (89%)	9925 ± 2504 (100%)

Function	PSO	FA
Michalewicz	6922 ± 537 (98%)	2889 ± 719 (100%)
Rosenbrock	32756 ± 5325 (98%)	6040 ± 535 (100%)
De Jong	17040 ± 1123 (100%)	5657 ± 730 (100%)
Schwefel	214522 ± 1275 (97%)	7923 ± 524 (100%)
Ackley	23407 ± 4325 (92%)	4392 ± 2710 (100%)
Rastrigin	79491 ± 3715 (90%)	12075 ± 3750 (100%)
Easom	17273 ± 2929 (90%)	6082 ± 1690 (100%)
Griewank	55970 ± 4223 (92%)	10790 ± 2977 (100%)
Yang	19725 ± 3204 (98%)	5152 ± 2493 (100%)
Shubert	23992 ± 3755 (92%)	9925 ± 2504 (100%)

Conclusion

La nature ne résout pas les équations, elle trouve simplement des solutions.

Ce que les lucioles ont mis des millions d'années à perfectionner, nous commençons à peine à le comprendre. L'algorithme des lucioles nous rappelle ainsi que la simplicité et l'émergence collective peuvent vaincre la complexité, révélant peu à peu les secrets d'une intelligence bien plus ancienne que la nôtre.



**Merci De Votre
Attention !**

N'hésitez pas à poser vos questions